Building Agentic AI Applications with PostgreSQL as the Backbone

Memory systems, tool registries, MCP integration, and guardrails for production agents

Shayon Sanyal · Principal PostgreSQL Specialist SA · Lead, Agentic AI for Databases

PostgresConf 2026 · San Pedro (Level C) · Apr 21, 15:00 PDT

WHY ARE WE EVEN HAVING THIS CONVERSATION?

The minute you ship an agent, it stops being a Python script. It becomes seven systems.

Seven auth boundaries. Seven failure modes. Seven things to page on at 2am.

ABOUT ME

Shayon Sanyal

Principal PostgreSQL Specialist Solutions Architect

Tech Lead — Agentic AI for Databases

I help teams build production-grade systems on PostgreSQL — from vanilla RDS deployments to Aurora clusters running agentic workloads.

Lately most of that work lives at the intersection of agents, pgvector, and actually shipping them. Less hype, more JOINs.

linkedin.com/in/shayonsanyal

The problem

Most "production agent" stacks look like this:

- **Pinecone** for vectors
- **Redis** for session + cache
- **Postgres** for app state
- **DynamoDB** for audit logs
- **Temporal** for workflow state
- **SQS** for approval queues
- **AgentCore / LangGraph** for orchestration

Seven systems. Seven auth boundaries. Seven failure modes. Seven things to explain on-call at 2am.

THE THESIS

One database. Every role a production agent needs.

The LLMs live at the edges. Everything in between is SQL.

Why one database, not five

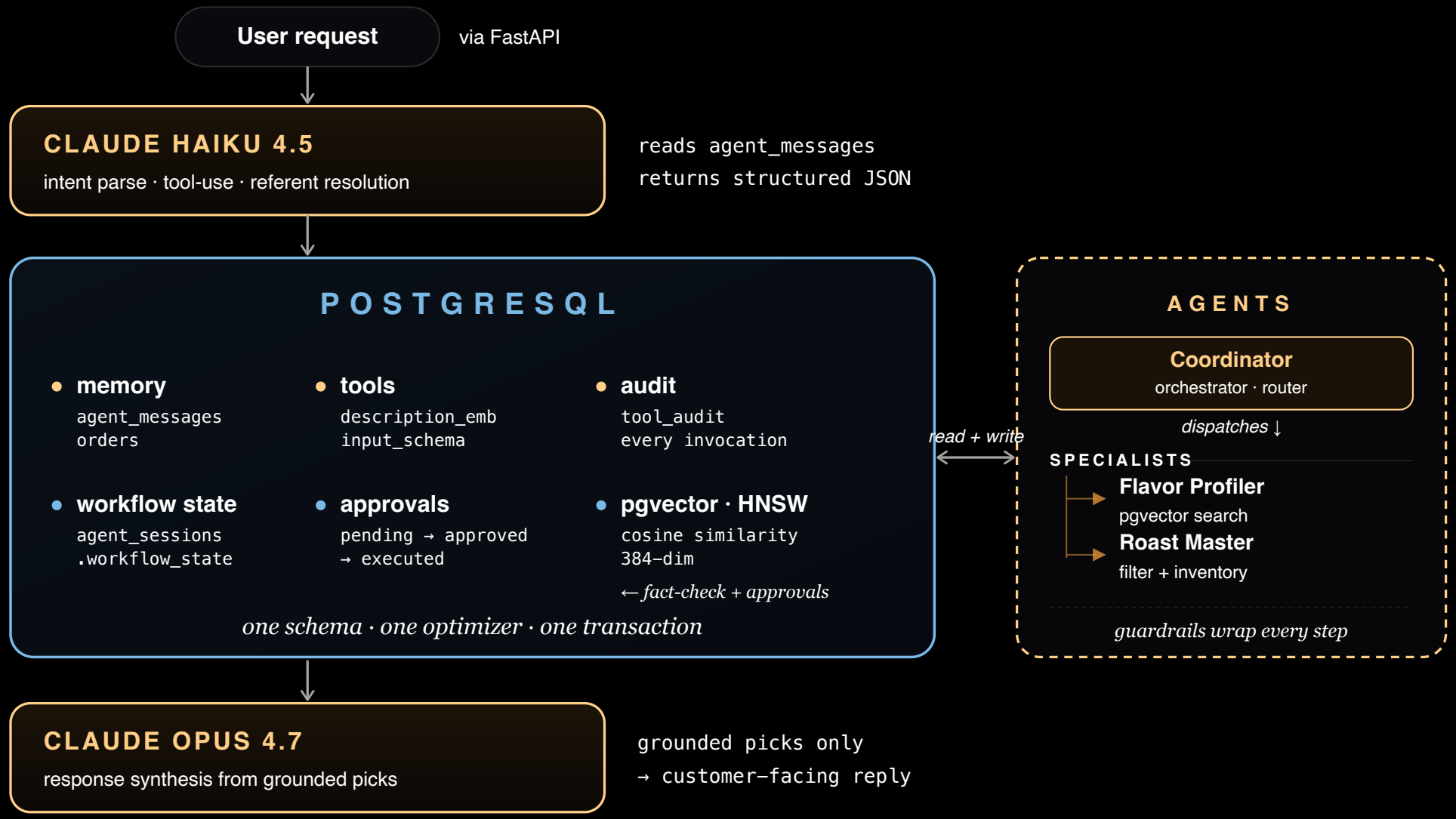
The usual argument: "you can't JOIN across Pinecone + Redis + DynamoDB + Postgres + Temporal."

True, but that's not the strongest claim. The strongest claim is:

- **One optimizer** sees vector similarity, row filters, and joins together and picks a plan
- **One transaction** commits the audit row, the approval, and the state checkpoint atomically
- **One MVCC snapshot** means your similarity scores and your inventory counts agree

Distributed systems people call this "avoid distributed consensus for state you don't need distributed." Postgres people call it Tuesday.

AI Coffee Roastery — Architecture



Two Claude models · Three agents · One Postgres DB

Haiku parses. Opus synthesizes.

STAGE	MODEL	ROLE
Intent parse + referent resolution	**Claude Haiku 4.5**	Reads last ~6 turns, returns structured JSON via Converse tool-use. Resolves "*order that*" → `b_espresso_blend`.
Response synthesis	**Claude Opus 4.7**	Receives only **grounded picks** + customer profile. System prompt locks it to cited bean ids. **Hallucination surface dramatically reduced — Opus can only reference the ids we pass in.**

Both via `bedrock-runtime.converse` in `us-east-1`. Every call → `tool_audit` with `tool = 'llm:<model_id>'`.

LLMs at the edges. Data in the middle. The contract (`intent → picks → citations`) is stable — swap either model tomorrow, nothing in Postgres changes.

Latency and cost per turn

Where time and money go in a single agent turn:

COMPONENT	LATENCY BAND	COST CONTRIBUTION
Haiku intent parse	hundreds of ms	low (small model)
Semantic tool lookup	single-digit ms	negligible
Procedural memory JOIN	~tens of ms	negligible
Fact-check reads	single-digit ms	negligible
Opus synthesis	seconds	**dominant**
Audit writes	single-digit ms	negligible
End-to-end turn	~a few seconds	effectively Opus

Exact latencies and Bedrock prices shift; shape of the distribution doesn't.

Headline: the database work is milliseconds; the LLM is seconds. The DB is the boring, cheap part.

Three memory types, one database

MEMORY	WHERE	ACCESS PATTERN
Episodic — conversation + actions	`agent_messages`, `orders`	`WHERE session_id=$1 ORDER BY ts DESC`
Semantic — domain knowledge	`beans.embedding vector(384)` + HNSW	`ORDER BY embedding <=> $1`
Procedural — learned patterns	`orders ⋈ beans ⋈ customers`	similarity + join in one query

Procedural memory: the zinger

```
WITH q AS (SELECT %s::vector AS v)
SELECT b.name, c.name AS customer, COUNT(*) AS times_ordered,
       MAX(1 - (b.embedding <=> (SELECT v FROM q))) AS similarity
  FROM orders   o
  JOIN beans    b ON b.id = o.bean_id
  JOIN customers c ON c.id = o.customer_id
 WHERE c.id <> %s
 GROUP BY b.name, c.name
 ORDER BY similarity DESC
 LIMIT 5;
```

Vector similarity + relational join + aggregation — one plan, one optimizer.

Procedural memory is the *relation between* episodic and semantic. It only exists when both live in the same database.

Tools are a table, not a list

```
CREATE TABLE tools (
  name              text PRIMARY KEY,
  description       text NOT NULL,
  description_emb   vector(384),   -- for semantic discovery
  input_schema      jsonb NOT NULL,
  requires_approval boolean NOT NULL DEFAULT false,
  enabled           boolean NOT NULL DEFAULT true,
  owner_agent       text
);
```

At query time: rank `description_emb` against the request → **dynamic discovery, not a static toolbox.**

Semantic tool discovery in one query

```
SELECT name, description,
       1 - (description_emb <=> $1::vector) AS score
  FROM tools
 WHERE enabled = true
 ORDER BY score DESC
 LIMIT 5;
```

The coordinator embeds the user's request, runs this, and picks tools above a score threshold (~0.55 for MiniLM-L6 in our setup).

Add a new tool → `INSERT` a row. Next request can discover it. **No code deploy, no prompt template change.**

`tool_audit` captures SQL tool calls *and* LLM calls as `tool = 'llm:<model_id>'`. One `SELECT` = complete execution trace. No four dashboards.

Same Postgres, different client

`mcp_server.py` — stdio MCP server exposing three tools:

- `list_tables` — schema introspection with approximate row counts
- `describe(table)` — columns + indexes, **allowlist-gated**
- `run_query(sql, params)` — **SELECT-only**, parameterized, DDL/DML rejected, 100-row cap
- **Enforcement**: MCP server connects as a read-only Postgres role (`GRANT SELECT ON ... TO mcp_reader`). SQL also parsed client-side to reject anything that isn't a single `SELECT`. Belt and suspenders.

Claude Desktop, Cursor, or any MCP host can query `agent_messages`, `tool_audit`, `approvals` directly.

Same Postgres, two interfaces. Agent writes via FastAPI. External LLMs read via MCP. No extra API to build, version, or secure.

Workflow state is a column

```
{
  "stage": "roast_master_filter",
  "step_index": 3,
  "query": "Cold brew options"
}
```

Checkpointed to `agent_sessions.workflow_state` (JSONB) after every step boundary.

Process dies mid-task? Next call resumes from `step_index`. **Same transaction as the side effects — the checkpoint can't drift.**

Durable state is a column, not a service. A JSONB column and a `COMMIT` is your checkpoint.

When the JSONB checkpoint stops being enough

Our agent loops are sub-minute. A JSONB column and `COMMIT` is perfect.

It stops being enough when you have:

- **Long-running workflows** (hours or days) with external callbacks
- **Complex compensation** — if step 4 fails, undo 1, 2, 3 in reverse with saga semantics
- **High-fanout parallelism** — hundreds of parallel steps per workflow
- **Cross-service coordination** where the workflow spans systems you don't own

That's where Temporal, Step Functions, or Cadence earn their keep. We're not claiming Postgres replaces them — we're claiming it replaces them *for the access pattern most agents actually have*.

Three layers keep Opus honest

1. **Fact-check pass** — every pick re-read from `beans` + stock-verified. Failures **dropped**, not papered over.
2. **Confidence from data** — row coverage, history match, top similarity. Not a fudge factor.
3. **Approval queue** — tools with `requires_approval=true` write a row to `approvals` with `status='pending'`. **Effect blocked until approved.**

Opus only sees the picks that survived all three.

Grounding is the guardrail Opus can't bypass. It can't reference what we don't put in its context.

INTERLUDE

Demo

Three agents, one Postgres, live.

FROM ARCHITECTURE TO PRODUCTION

Operational Realities

Autovacuum, pool sizing, and HNSW knobs.

pgvector: HNSW tuning

```
CREATE INDEX ON beans USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Query-time recall knob:
SET hnsw.ef_search = 40;
```

Three knobs: `m` (graph connectivity), `ef_construction` (build-time recall), `hnsw.ef_search` (query-time recall). First two are baked in at build — tune `ef_search` per query.

EF_SEARCH	RECALL @10	QUERY LATENCY
low (~20)	~0.90	sub-ms floor
default (40)	~0.95	low single-ms
high (~80)	~0.98	mid single-ms
very high	~0.99+	approaches 10ms

Numbers vary by corpus, dimension, and hardware — treat as shape, not spec.

The curve matters more than any single row: recall climbs fast, then saturates; latency climbs linearly. For most agents, the default is already on the flat part of the recall curve.

Append-heavy tables: autovacuum earns its keep

`agent_messages` and `tool_audit` are write-mostly. What goes wrong:

- HOT updates don't help — these rarely UPDATE
- Autovacuum kicks in on dead-tuple threshold, which you won't hit; you need it to run on **insert** threshold too
- TOAST bloat on long content columns; set `toast_tuple_target` sensibly
- HNSW indexes fragment on insert

Concrete settings we run:

```
ALTER TABLE agent_messages SET (
  autovacuum_vacuum_insert_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.02,
  fillfactor = 90
);
```

Connection pooling: agents are chatty

A single agent turn fires 8–15 SQL queries (message reads, tool discovery, fact-check, procedural JOIN, audit writes, checkpoint). At 50 concurrent sessions, ~500 queries in flight.

Don't point those at raw backend connections.

- **PgBouncer** in transaction mode, `max_client_conn = 1000`, `default_pool_size = 25`
- Prepared statements work in transaction mode as of PgBouncer 1.21+ — turn this on
- **RDS Proxy** if you're on AWS and don't want to own PgBouncer

Agents don't need a new database. They need a tuned one — autovacuum knobs, pool sizing, HNSW parameters. Boring, knowable, Postgres.

Chat continuity — "order that"

Turn 1: "*Cold brew options*" → Opus recommends **House Espresso Blend**.
Turn 2: "*Order a bag*" → ??

The problem: re-embedding turn 2 lands on a different top hit (`Honduras Marcala`). Customer sees a bait-and-switch.

The fix: Haiku's `record_intent` tool-use schema includes `order_referent_bean_id`. It reads the last six turns of `agent_messages` and returns the bean id from the previous recommendation. Coordinator pins that id to the front of `verified`. Approval row + Opus confirmation both reference the **same bean the customer was pitched**.

The LLM and the coordinator share a memory. That memory is a boring SQL table.

When NOT to do this

Postgres is not the answer for every agent stack. Honest limits:

- **Billion-scale vectors** — pgvector HNSW builds slow down past ~10M rows per index. Look at **pgvectorscale** (StreamingDiskANN on top of Postgres), **DiskANN-style** indexes, or Google's **ScaNN** — and if you need more, specialized vector stores (Milvus, Qdrant, Vespa) beat you on build time and ANN latency.
- **Sub-millisecond p99** — agent in a trading hot path? A purpose-built in-memory KV (Redis, DragonflyDB) beats Postgres.
- **Workflows measured in days** — saga compensation, human-in-the-loop over SLAs, cross-system orchestration. Temporal exists for a reason.
- **Hard multi-tenant isolation at scale** — thousands of tenants with strong blast-radius requirements. RLS + partitioning works, but separate clusters are eventually cleaner.
- **Team has zero Postgres operators** — "just use Postgres" assumes someone knows how to operate it. If your platform team only does DynamoDB, factor that in.

For everything else — which is most agents most of the time — one database wins.

Every role, one plan

```
SELECT b.name, b.roast_level, b.in_stock,
       1 - (b.embedding <=> (SELECT embedding
                              FROM beans
                             WHERE id='b_ethiopia_guji')) AS similarity,
       (SELECT count(*) FROM orders o
         WHERE o.bean_id = b.id AND o.customer_id = 'u_marco')
         AS marco_has_bought,
       (SELECT count(*) FROM tool_audit ta
         WHERE ta.result->'stock' ? b.id)
         AS times_inventory_checked,
       (SELECT count(*) FROM approvals a
         WHERE a.args->>'bean_id' = b.id AND a.status='pending')
         AS pending_orders
  FROM beans b
 WHERE b.in_stock > 0
   AND b.roast_level IN ('medium','medium-dark','dark')
 ORDER BY similarity DESC LIMIT 5;
```

One plan. One optimizer. One transaction. Draw that when your vectors are in Pinecone, state is in Postgres, and your audit is in DynamoDB.

Where this pattern lands in practice

We see teams landing on variants of this pattern in production — across customer environments on Aurora PostgreSQL:

- Multi-agent customer-facing assistants with approval queues
- Internal ops agents that read operational telemetry and propose remediations
- Migration assistants that read legacy schemas and generate Postgres DDL

The data layer looks nearly identical across all three — `tools`, `tool_audit`, `approvals`, `agent_messages`. Domain tables change. **The backbone doesn't.**

And it plays nicely with frameworks. LangChain, LangGraph, Strands, AgentCore — most teams land on one. Postgres doesn't fight them. LangGraph's `PostgresSaver` is a wrapper over a JSONB column; Strands memory can point at a Postgres table; AgentCore's session store speaks SQL. **Frameworks handle prompt orchestration and agent loops; Postgres handles state, memory, audit, approvals. Clean seam.**

Customer names stay off the slides — ask me after and I'll share what I can.

Architecture reference

PILLAR	STORAGE / COMPONENT	NOTE
LLM orchestration	Haiku 4.5 on Bedrock	Intent + referent resolution via tool-use
LLM synthesis	Opus 4.7 on Bedrock	Grounded response, cited against `beans.id`
Episodic memory	`agent_messages`, `orders`	Indexed by `(session_id, ts DESC)`
Semantic memory	`beans.embedding` + HNSW	pgvector cosine similarity
Procedural memory	`orders ⋈ beans ⋈ customers`	Cohort few-shot context — one query
Tool registry	`tools`, `tools.description_emb`	JSON schemas + semantic discovery
Tool + LLM audit	`tool_audit`	SQL calls + `tool='llm:…'`, same table
Workflow state	`agent_sessions.workflow_state`	JSONB checkpoint, resumable
Approvals	`approvals`	Blocks sensitive tools until approved
MCP surface	`mcp_server.py`	list_tables / describe / run_query

Run it yourself

```
git clone https://github.com/shayons/talks.git
cd talks/conferences/2026-postgresconf-agentic-ai
createdb coffee
psql coffee -f schema.sql

python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

export AWS_REGION=us-east-1
python seed.py    # first time only
python app.py     # → http://localhost:8000
```

Needs **pgvector 0.5+** (for HNSW) and AWS credentials with Bedrock access.

Thank you

Questions?

Repo + slides · github.com/shayons/talks · PostgresConf 2026